

Rapport de projet INFO1b : Jeu des 7 couleurs

Julie MESUREUX

Remarque : Les explications sur le fonctionnement interne des programmes sont données en commentaire dans ces derniers.

Remarque : Nous avons finalement pas pu travailler en groupe (logistique trop compliquée), j'étais à l'origine groupe 4, j'ai renommé tous mes programmes avec "GP4_b_...".

Première partie

Projet

1 Création du jeu

Afin de créer un jeu de fonctionnel, il faut tout d'abord gérer la création, l'affichage et les modifications du plateau de jeu, ainsi que la désignation du vainqueur.

Dans le squelette fourni, on trouve la définition de deux objets essentiels pour le projet. Le premier est une énumération nommée `Color` permettant de faciliter le stockage des différents états possibles pour une case en les liant chacun à un entier. Le second est la structure `GameState`, cette dernière servira à stocker l'adresse du plateau de jeu dans la mémoire, ainsi que sa taille.

Le plateau est stocké dans la heap. La fonction `create_empty_game_state` prend en argument un pointeur vers une structure de type `GameState` pour stocker les informations importantes relatives au plateau, et un entier représentant la largeur souhaitée du plateau. Elle permet d'allouer l'espace nécessaire pour le plateau à l'aide d'un `malloc` et de mettre à jour la structure `GameState`. Le plateau de jeu est un tableau 1D (indexé de 0 à n où n représente le nombre de cases moins un) mais on l'utilisera comme un tableau 2D (avec n lignes et n colonnes où n est la taille donnée pour la création du plateau). On passe alors de l'un à l'autre avec les formules suivantes :

$$i = x + size \times y$$
$$\begin{cases} x \equiv i[size] \\ y = \frac{i-x}{size} \end{cases}$$

Les coordonnées 2D x et y commencent à 0 : x représente la colonne et y la ligne.

On doit ensuite remplir ce plateau de jeu par des couleurs et mettre en place les joueurs. On crée tout d'abord la fonction **set_map_value** qui permet d'assigner la couleur passée en argument à la case de coordonnées (x, y) fournies. Cette fonction est ensuite utilisée dans une fonction nommée **fill_map** qui remplit le plateau de jeu créé avec des couleurs aléatoires et place les joueurs à leur emplacement respectif sur le plateau. La génération aléatoire d'un entier entre 3 et 9, correspondants aux sept couleurs du jeu, est effectuée grâce à la fonction **rand** du module **stdlib.h**. J'ai fait le choix de remplir tout le plateau avant de placer les joueurs.

Dans la fonction **main**, on initialise une partie en lisant la taille souhaitée dans les arguments d'appels du programme. Si la taille n'a pas été fournie, elle sera demandée dans le terminal. Pour ne pas causer de fuite de mémoire, il ne faudra pas oublier de libérer l'espace mémoire alloué au plateau de jeu une fois que la partie est terminée à l'aide d'un **free**.

Pour qu'il soit possible pour un humain de jouer, il faut pouvoir afficher l'état du plateau, pour cela j'ai définie la fonction **GP4_b_print_state**. Afin d'afficher la lettre correspondant à la couleur de la case, il a fallu faire différents blocs if, un pour chaque couleur. Les joueurs sont représentés par un 1 ou un 2.

Il est également nécessaire de faire une fonction qui viendra mettre à jour le territoire d'un joueur donné en fonction de la couleur qu'il souhaite annexer. La fonction **GP4_b_MAJ_plateau_non_opti** permet cette modification du plateau, la méthode utilisée est celle proposée dans le sujet, elle n'est pas optimisée et nécessite de parcourir le plateau en entier plusieurs fois. Pour faciliter le parcours, notamment au niveau des bords, j'ai créé la fonction **GP4_b_return_map_value**, inspirée de la fonction fournie **get_map_value**, qui permet de retourner la couleur d'une case ou EMPTY si la case est en dehors du plateau.

Enfin, il faut pouvoir déterminer s'il y a un vainqueur, c'est-à-dire si un joueur a envahi au moins 50% du plateau de jeu. C'est le rôle de la fonction **GP4_b_fin_de_partie** qui vient compter le nombre de cases possédées par le joueur désigné et de le compare avec le nombre total de cases du plateau. Si le joueur ne remplit pas la condition de victoire, la fonction renvoie EMPTY, sinon elle retourne la « couleur » du joueur.

Remarque : le (float) permet au résultat de la division de deux entiers d'être un flottant. J'utilise également cette méthode plus loin dans la fonction permettant de faire une compétition entre les IA.

2 Partie à deux joueurs

Une fois les fonctions nécessaires au fonctionnement du jeu créées, on peut désormais mettre en place une partie entre deux joueurs. Pour cela, on crée une première fonction **GP4_b_play** qui permet d'effectuer le tour d'un joueur. Elle prend en argument le plateau, le joueur et la couleur à jouer. En prenant en argument la couleur à jouer, on

peut utiliser cette fonction lorsque l'on joue avec une IA, pour un joueur réel on appelle la fonction avec la couleur EMPTY.

J'ai eu beaucoup de mal à faire fonctionner ce programme car le tour du joueur 1 était sauté à chaque fois. Après quelques recherches, j'ai trouvé la source du problème¹, les **scanf** à la suite n'étaient pas bien lus si j'utilise "%d", il faut ajouter un espace avant "%d".

3 "Intelligences artificielles"

Nous allons désormais programmer quelques intelligences artificielles que nous pourrions affronter ou faire s'affronter. Ces IA sont des fonctions prenant en argument le plateau de jeu et le joueur et qui renvoie le coup qui respecte sa stratégie de jeu. On fera s'affronter ces IA pour connaître celle ayant la meilleure stratégie.

3.1 Je fais n'importe quoi

Cette première IA, **GP4_b_fait_n_importe_quoi**, joue une couleur au hasard. Le problème que j'ai rencontré est que le temps d'exécution entre chaque tour est tellement court que la seed de génération d'aléatoire est identique pour un certain nombre de tours à la suite et donc les mêmes coups sont répétés. Pour pallier ce problème, j'ai créé une variable statique m qui est un simple entier incrémenté de 1 à chaque appel de la fonction. Afin d'obtenir les valeurs aléatoires, je répète m fois **rand** avant de tirer au hasard la couleur ce qui me permet d'avoir à chaque fois la valeur suivante dans la suite de nombres aléatoires générés. Cette manière de faire n'est certainement pas optimale dans le cas où la partie s'éternise, j'ai imposé une limite de 25 à m, pour que la boucle for n'explose pas, de plus assez de temps devrait s'être écoulé pour que la seed est changée au bout de 25 appels de la fonction.

Remarque : le problème ne se pose pas dans la fonction fill_map car les nombres aléatoires sont tous générés à la suite lors du même appel de la fonction.

Une amélioration de ce joueur artificiel est l'IA **GP4_b_aleatoire**. Cette dernière joue également une couleur aléatoire, mais la couleur choisie doit forcément être une couleur qui est en contact avec le territoire du joueur. On va donc tirer aléatoirement une couleur mais en ayant auparavant parcouru le plateau entièrement pour établir un tableau avec les couleurs adjacentes au territoire du joueur. On ne retourne la couleur aléatoire qu'après s'être assuré qu'elle est dans le tableau. J'ai utilisé la même stratégie que précédemment pour régler le problème de tirage aléatoire.

3.2 Glouton

Le joueur artificiel **GP4_b_glouton** a pour objectif de jouer la couleur qui lui permet d'annexer le plus de cases possibles. Pour cela, on effectue pour chaque couleur une

1. <https://openclassrooms.com/forum/sujet/la-fonction-scanf-ne-marche-pas-deux-fois-de-suite>

copie du plateau de jeu, puis on joue des tours artificiels afin de voir ce que donnerait l'annexion de chaque couleur. On compte le nombre de case du joueur après modifications pour comparer le nombre de cases qui composeraient le territoire du joueur en fonction de la couleur jouée. On retourne la couleur qui permet d'annexer le plus grand nombre de case.

3.3 Hégémonie

L'IA **GP4_b_hegemonique** a pour objectif d'annexer la couleur qui augmente le plus son nombre de cases à la frontière du territoire. Ces cases sont importantes car, étant adjacente à au moins une case de couleur, elle permettent au joueur de pouvoir continuer d'agrandir son territoire au coup suivant. Pour désigner le coup à jouer, on utilise la même stratégie que pour le joueur artificiel précédent mais cette fois ci on ne compte plus toutes les cases du joueur, mais uniquement celles qui sont en contact avec au moins une case couleur.

Cette intelligence artificielle a un énorme problème, elle a tendance à rester bloquée si l'annexion des couleurs voisine de son territoire font diminuer le nombre de case à sa frontière (Elle joue alors en boucle une couleur qui n'est pas en contact avec son territoire). En particulier, le problème se pose lorsque le territoire des deux joueurs se rejoignent, à ce moment là la partie est bloquée et ne finira jamais si seul le joueur hégémonique à la possibilité d'y mettre fin.

3.4 Mixte

Ce dernier joueur artificiel, **GP4_b_mixte**, permet en parti de corriger le défaut évoqué précédemment. En effet ce joueur à la même stratégie que l'IA hégémonique, maximiser ses frontières, mais adopte une stratégie de glouton si aucune des couleurs ne lui permet cela. Pour cela, on vient calculer le nombre de cases initialement à la frontière. En soustrayant ce nombre au nombre de cases à la frontière après modification, cela permet de savoir si une couleur augmente (nombre positif) ou diminue (nombre négatif ou nulle) les frontières. Si aucune des couleurs augmente les frontières, on applique la stratégie glouton.

4 Tournois

Afin de faire s'affronter les IA est connaître la meilleur stratégie, j'ai créé la fonction **GPA_competition** dont j'explique le rôle un peu plus bas dans ce rapport.

Remarque : Je n'ai pas pu inclure l'IA hégémonique dans la compétition car le défaut évoqué ci-dessus cause toujours une des partie de se bloquée et la compétition ne se finie jamais.

Je souhaite tout d'abord valider l'hypothèse que le joueur 1 est toujours avantage. Pour cela, je fait s'affronter les même IA entres elles. Pour que les statistiques soient valident, j'effectue 1000000 affrontements à chaque fois (Figure 1).

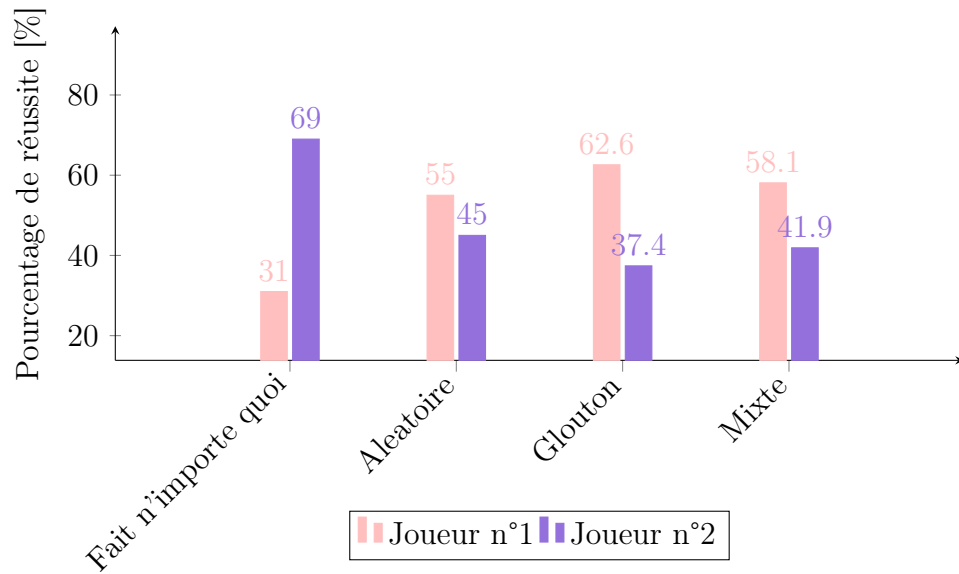


FIGURE 1: Avantage offert par la position de Joueur 1 (10×10)

On se rend compte que, à l'exception du joueur artificiel jouant totalement au hasard, tous les autres joueurs artificiels montre un avantage pour le joueur 1.

Afin de comparer les IA je vais tout d'abord les faire s'affronter 500000 fois sur un plateau de 10×10 . Chacune affrontera toutes les autres en tant que joueur 1 et en tant que joueur 2 (Soit 1000000 parties en tout pour chaque paire d'IA).

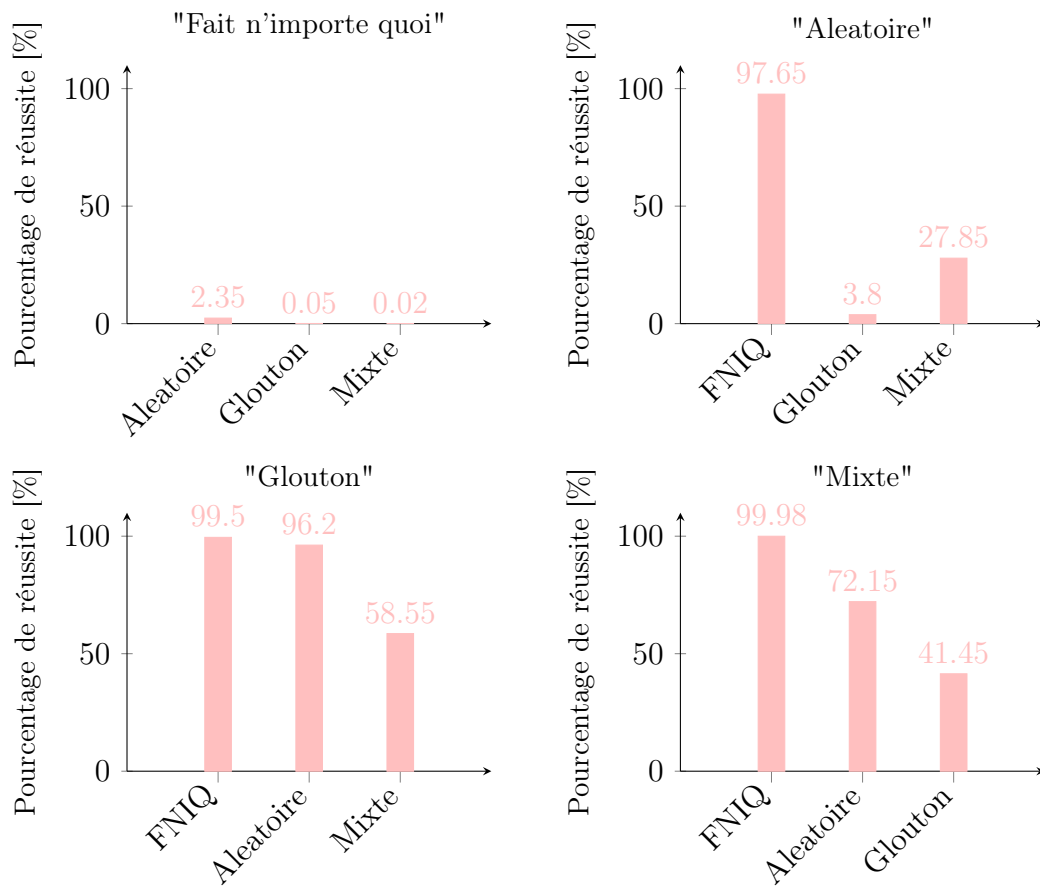


FIGURE 2: Détails du taux de réussite de chaque IA sur un plateau 10×10

On voit tout d'abord que l'IA jouant totalement au hasard n'a aucune chance face aux autres. Le joueur Aléatoire n'est vraiment efficace que contre l'IA évoquée précédemment. Enfin, contre toute attente, la stratégie gloutons semble être la plus avantageuse, même face à la stratégie plus élaborée de l'IA Mixte. Cette conclusion apparaît sur le figure 3 ci-dessous, l'IA glouton est bien celle assurant un plus grand pourcentage de réussite.

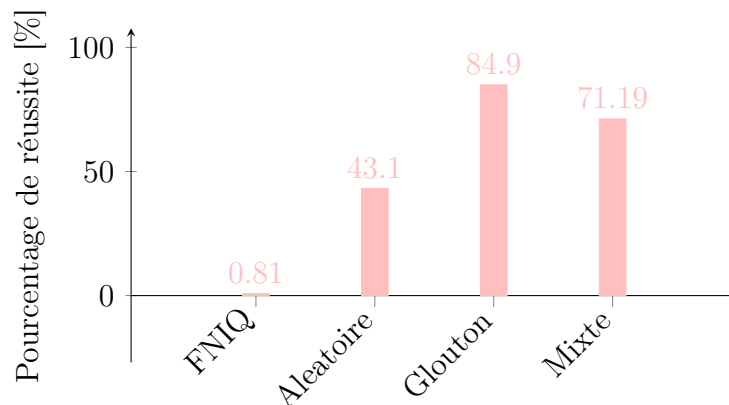


FIGURE 3: Moyenne du taux de victoire (10×10)

Pour regarder si la taille du plateau a un impact sur la stratégie à adopter, il est intéressant de reproduire l'expérience ci-dessus en utilisant un petit plateau (5×5) (Figures 4 et 5) et un grand plateau (20×20) (Figures 6 et 7).

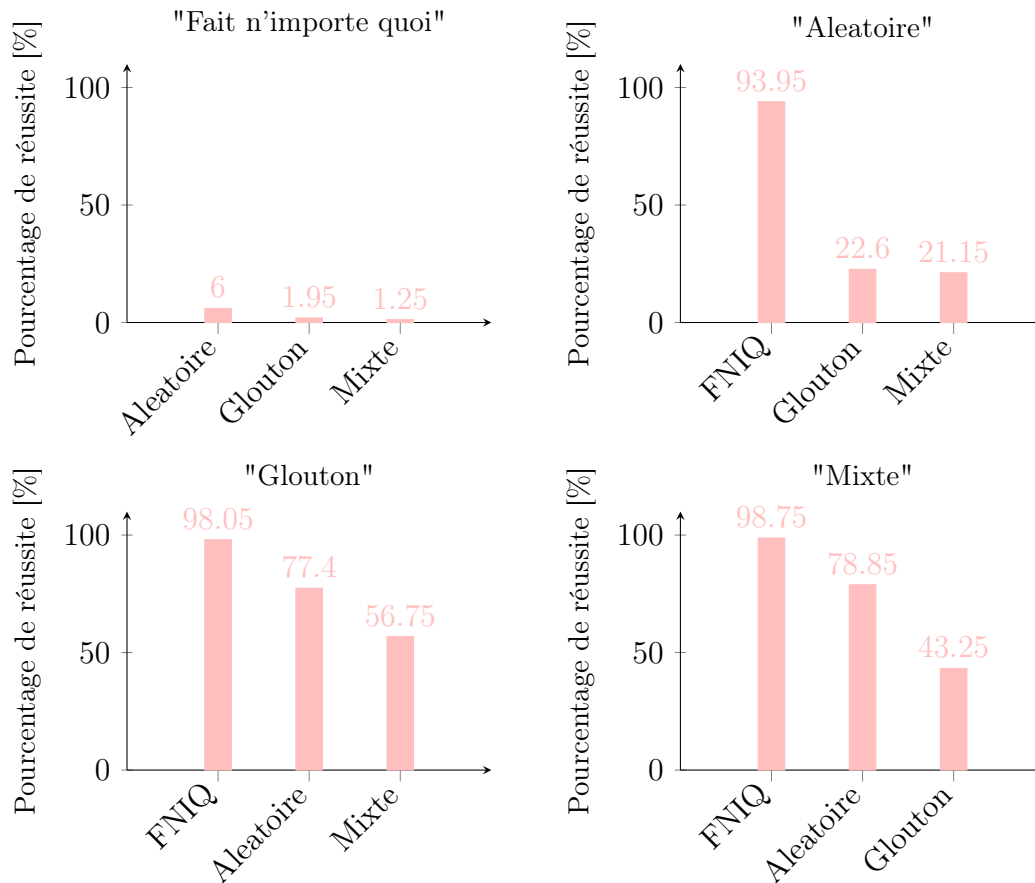


FIGURE 4: Détails du taux de réussite de chaque IA sur un plateau 5×5

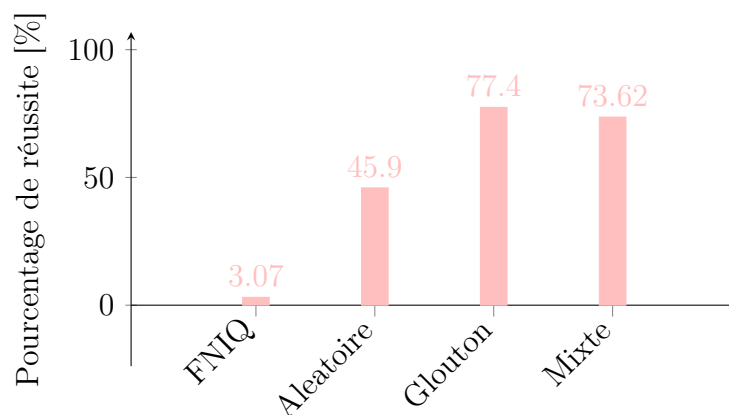
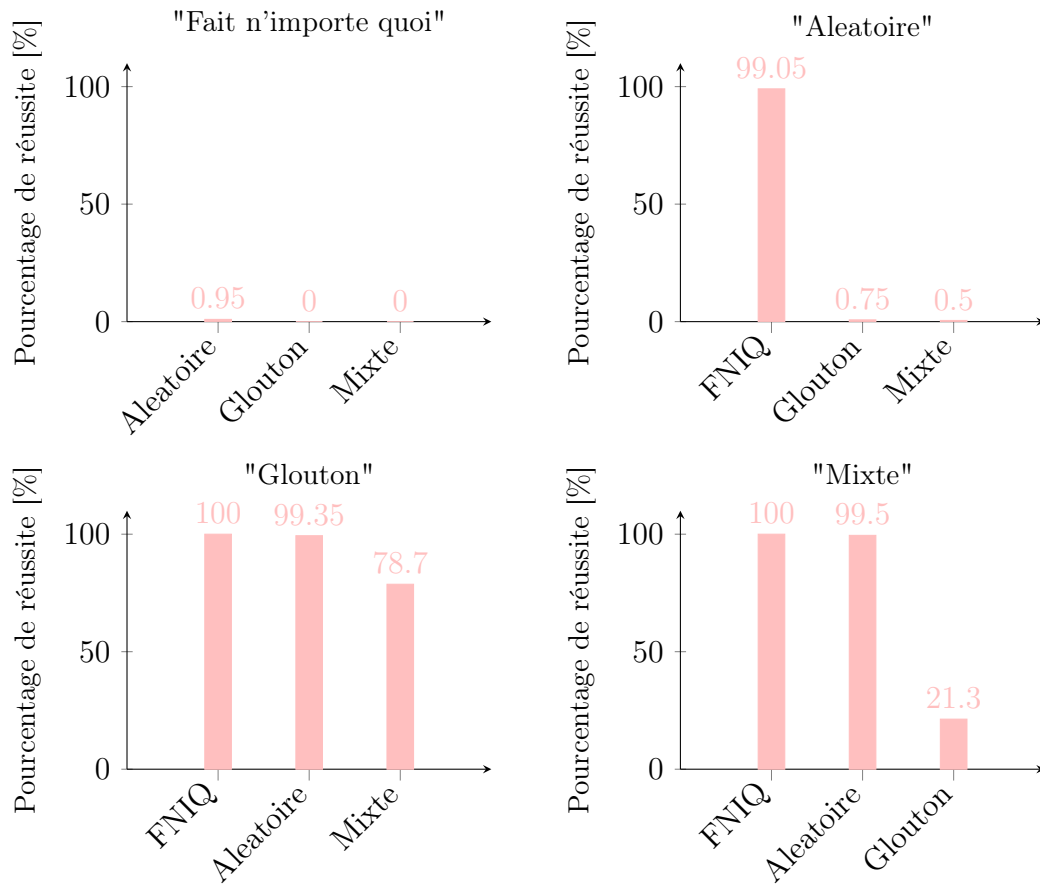
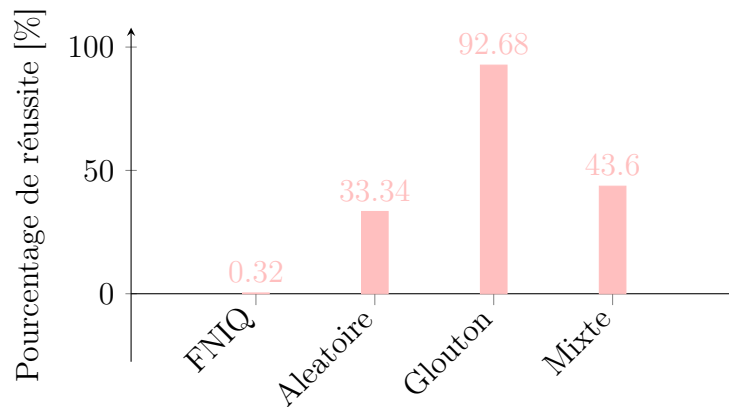


FIGURE 5: Moyenne du taux de victoire (5×5)

FIGURE 6: Détails du taux de réussite de chaque IA sur un plateau 20×20 FIGURE 7: Moyenne du taux de victoire (20×20)

Sur un plateau assez petit, la stratégie utilisé a moins d'importance. Au contraire, lorsque les dimensions du plateau sont assez grandes, l'écart entre les stratégies se creuse et l'IA Glouton s'impose comme étant la plus efficace.

Deuxième partie

Bonus et améliorations

5 Amélioration de l’affichage

Pour améliorer l’affichage du plateau de jeu dans le terminal, j’ai créé la fonction **GP4_b_affichage_couleur**. Cette dernière permet, en plus d’afficher les lettres, d’afficher les cases en couleurs². J’ai tout de même conservé la lettre correspondante à chaque couleur pour garder la même manière pour un joueur réel de donner la couleur qu’il souhaite annexer.

6 Optimisation de la mise à jour du plateau de jeu

Remarque : tout ce qui concerne cette partie est mis en place dans une version alternative ./?color_opti dans le dossier Optimisation qui ne respecte plus toutes les contraintes du projet.

6.1 Mise à jour

J’ai souhaité tenter d’optimiser le parcours du plateau. Pour cela, j’ai dû créer des fonctions de gestion de liste chaînée. Ces listes chaînées me permettront de mémoriser les cases qui sont à la frontière du territoire de chacun des joueurs dans un tableau dynamique. Ainsi, l’objectif est de ne traiter que les cases de couleurs qui sont à côté des frontières.

Cette nouvelle méthode de modification **GP4_b_MAJ_plateau**, s’inspire du parcours de graphe en profondeur. J’initialise une pile temporaire (LIFO) où je copie les cases de ma frontière avant mise à jour. C’est une liste chaînée donc pour minimiser la complexité de parcours, je ne regarde que le premier élément, d’où le parcours en profondeur. Pour chaque case de la liste temporaire, je la supprime de la liste, je regarde les voisins, je modifie ceux de la couleur annexée, puis les ajoutent à ma pile et à ma liste des cases de la frontière. Quand ma pile est vide, la mise à jour du plateau est terminée, il faut ensuite vérifier si les cases contenues dans la liste des frontières sont toujours des frontières. Ici étant donné qu’il est nécessaire de parcourir la liste entièrement j’ai créé une fonction spéciale **GP4_b_sup_if_not_front**. Cette dernière permet de traiter les suppressions en un seul parcours de la liste.

Enfin, je parle depuis le début de cette sous-partie de l’identification d’une frontière. La fonction **GP4_b_est_une_frontiere** permet cela en regardant si la case indiquée est à côté d’une case couleur. Si ce n’est pas le cas, ce n’est pas une frontière, il n’est donc plus la peine de la regarder.

2. <http://sdz.tdct.org/sdz/des-couleurs-dans-la-console-linux.html>

6.2 Utilisation pour améliorer les IA

Cette d'optimisation de la mise à jour du plateau permet d'accélérer la plupart des intelligences artificielles, le problème est qu'elles ne respectent plus les conditions définies dans le sujet (il est nécessaire de leur fournir la liste des frontières du joueur en plus). Je ne m'en suis donc servi uniquement pour effectuer la comparaison des IA car la diminution du temps de calcul m'a permis de faire un grand nombre de parties et ainsi obtenir des statistiques plus fiables.

On peut remarquer que pour les IA Hégémoniques et Mixte, la méthode choisie pour l'optimisation de la modification du plateau permet de grandement faciliter leur programme. En effet, une fois la copie du plateau mise à jour, il suffit de compter le nombre de cases dans la liste des frontières.

Remarque : sur un grand nombre de parties, il m'arrive d'avoir un "Segmentation fault" j'en ai cherché l'origine pendant longtemps sans succès. Vu la fréquence très basse de cette erreur, il doit y avoir un cas particulier que j'ai oublié de prendre en compte.

7 Interface de jeu

Pour faciliter et améliorer l'utilisation du jeu des 7 couleurs, j'ai voulu créer un moyen de naviguer entre les différents modes de jeu. Pour cela j'ai décomposé les différents cas suivant :

Deux joueur : les utilisateurs fournissent la taille du plateau et une partie à deux joueurs se lance.

Un joueur : l'utilisateur choisit la taille du plateau et contre quelle IA il souhaite jouer. L'utilisateur joue toujours en tant que joueur 1. Ce mode de jeu m'a été très utile pour tester les différentes IA.

Zéro joueur : l'utilisateur choisit la taille du plateau et les deux IA qu'il souhaite faire s'affronter (IA n° = joueur 1 / IA n°2 = joueur 2). Il est possible de les faire s'affronter un certain nombre de fois et d'obtenir le résultat de ces affrontements.

Pour naviguer entre les différents modes de jeu on peut soit le faire directement à l'appel du programme :

`./7color <taille du plateau> <nombre de joueur> <IA n°1> <IA n°2> <nombre d'affrontements>`

Les arguments sont optionnels mais si on souhaite en donner un, il faut également préciser tous les précédents. On ne peut pas donner le nombre de joueur sans avoir également fourni la taille du plateau de jeu par exemple. Les arguments doivent être mis sous la forme suivante :

Taille du plateau → entier positif strictement.

Nombre de joueurs → entier positif parmi {0, 1, 2}.

IA n°... → entier positif {0 = fait n'importe quoi 1 = aléatoire 2 = glouton 3 = hégémonique 4 = mixte}

Nombre d'affrontements → entier strictement positif, peut être aussi grand que l'on souhaite dans la limite de la puissance de calcul disponible.

On peut également uniquement appeler `./?color` et fournir les valeurs demandées dans la console.

Pour chacun des modes de jeu, une fonction est associée : **GP4_b_two_players_game**, **GP4_b_one_player_game** et **GP4_b_AI_game**. Toutes ces fonctions utilisent la même fonction, **GP4_b_play**, pour l'affichage et la mise à jour du plateau. Pour les compétitions, j'ai créé la fonction **GP4_b_competition**, elle fait appel à la fonction **GP4_b_AI_game** pour effectuer le nombre de parties voulu. Elle permet également de gérer la création d'un nouveau plateau de jeu entre chaque partie et de comptabiliser le nombre de victoire de chacun des joueurs, le pourcentage de victoire de chacun est affiché à la fin de la compétition.